

As you can recall from high school math, typically real-world processes are described using real numbers, denoted by $\mathbb{R}$. For example, to describe the position or the velocity of an object, you would rather use real numbers instead of fields, groups, or p-adic numbers.

However, when it comes to working with computers, real numbers become very inconvenient to work with. The primary reason is that the set $\mathbb{R}$ is uncountably infinite, so the computer can only store the approximation of a real number (as the precise interpretation of the real number would require the infinite number of bits). For instance, different programming languages might output different values for quite a straightforward operation, such as an addition $2.00001 + 2.00000$. This becomes a huge problem when dealing with cryptography, which must check *precisely* whether two quantities are equal.

This is why the cryptography must work with integer-based formats. One of the earliest and most fundamental branches of mathematics is number theory, which deals with the properties of the integer set $\mathbb{Z}$. Although, as we will see later, we will primarily need notions of *finite fields* and *groups* further, the basic concepts of number theory will be used extensively nonetheless.

0.1 Division, GCD, LCM

We start with the most basic definition of number theory — *divisibility*.

Definition 0.1. An integer a is divisible by a non-zero integer b , denoted $b \mid a$ (or b is a *divisor* of a), if and only if there exists an integer $k \in \mathbb{Z}$ such that $a = k \cdot b$.

Example 0.1. Let us consider the following examples,

1. $41 \mid 123$, since $123 = 3 \cdot 41$
2. $5 \nmid 29$, since there is no integer k such that $29 = k \cdot 5$

Let us now explore some more basic properties of divisibility.

Lemma 0.2 (Divisibility properties). For all $a, b, c \in \mathbb{Z}$:

1. $1 \mid a$ (any number is divisible by 1)
2. If $a \neq 0$, then $a \mid a$ (any non-zero integer divides itself).
3. If $a \neq 0$, then $a \mid 0$ (any non-zero integer divides 0).
4. If $b \mid a$ and $c \mid b$, then $c \mid a$ (formally called *transitivity*).
5. $b \mid a \Leftrightarrow b \cdot c \mid a \cdot c$ for any $c \neq 0$.
6. If $c \mid a$ and $c \mid b$, then $c \mid (\alpha \cdot a \pm \beta \cdot b)$, for any $\alpha, \beta \in \mathbb{Z}$

This way, if we have a and b with $b \mid a$, we can safely divide a by b . However, in the majority of cases, it so happens that a is not divisible by b , making it hard

to define the division in such cases. For that reason, consider one of the central theorems of Number Theory: the *Division theorem*.

Theorem 0.3 (Division theorem). For any integers $a, b \in \mathbb{Z}$ there exist unique integers $q, r \in \mathbb{Z}$ with $0 \leq r < |b|$ such that

$$a = b \cdot q + r.$$

This theorem allows us to define two new operations. Suppose a, b, q, r are given as in the **Definition 0.3**. Then,

- **Floor Operation** ($\lfloor a/b \rfloor$ or $a \text{ div } b$) is defined as q . This operation is a standard div operation commonly used in programming languages.
- **Mod Operation** ($a \bmod b$) is defined as r . This operation is a standard mod operation commonly used in programming languages.

Remark. There is one caveat with programming languages, though. Often-times, the quotient q and remainder r obey the aforementioned condition $a = b \cdot q + r$, but with the difference that $|r| < |b|$, allowing two possible values for r , including negative.

Example 0.2. Continuing **Example 0.1**,

1. We know that $5 \nmid 29$. However, by **Definition 0.3** we can express 29 as $29 = 5 \cdot 5 + 4$.
2. Similarly, it is evident that $34 \nmid 123$, but $123 = 3 \cdot 34 + 21$.

To compare the results, we can use the following Python code:

```
def divmod(a: int, b: int) -> Tuple[int, int]:
    q = a // b
    r = a % b
    return q, r

a, b = 29, 5
q, r = divmod(a, b)
assert a == b * q + r, 'Theorem does not hold'

print(q, r) # (5, 4)
```

In division operations, it is common to check for any shared factors between two numbers. This is where the concept of the **greatest common divisor** (or **GCD** for short) comes into play.

Definition 0.4 (GCD). For any $a, b \in \mathbb{Z}$, the **greatest common divisor** $\gcd(a, b)$ is defined as an integer $d \in \mathbb{N}$ such that:

1. $d \mid a$ and $d \mid b$.
2. d is a maximal integer that satisfies the first condition.

One might write the above definition more concisely:

$$\gcd(a, b) = \max\{d \in \mathbb{N} : d \mid a \text{ and } d \mid b\}.$$

Example 0.3. Just as an example, let us find the $\gcd(a, b)$ for a few pairs of integers.

1. $\gcd(42, 28) = 14$
2. $\gcd(36, 14) = 2$
3. $\gcd(13, 17) = 1$
4. $\gcd(13, 18) = 1$

Definition 0.5. Integers $a, b \in \mathbb{Z}$ are called **coprime** if $\gcd(a, b) = 1$.

As with any fundamental concept, let us explore some basic properties of the GCD operation.

Lemma 0.6 (Greatest common divisor properties).

1. $\gcd(a, b) = b \Leftrightarrow b \mid a$
2. If $a \neq 0$, then $\gcd(a, 0) = a$
3. If there exists $\delta \in \mathbb{Z}$ such that $\delta \mid a$ and $\delta \mid b$, then $\delta \mid \gcd(a, b)$
4. If $c > 0$, then $\gcd(ac, bc) = c \cdot \gcd(a, b)$
5. Integers $a/\gcd(a, b)$ and $b/\gcd(a, b)$ are coprime.

Now, from the programming standpoint, you might ask how to implement the GCD computation in practice. Consider the following lemma which, as of now, might seem quite abstract.

Lemma 0.7. For any integers $a, b \in \mathbb{Z}$, we have $\gcd(a, b) = \gcd(b, a - b)$.

How does it help? Here is how: instead of computing $\gcd(a, b)$ directly, we reduce the problem to a simpler one, where one of the numbers is smaller. We apply this recursively until, at some point, one of the integers is zero. The other one will be the GCD result, as follows from the second property of [Definition 0.6](#). Surprisingly, this can be written in a couple of lines of code:

```
def gcd(a: int, b: int) -> int:
    # We want a to be always greater than b
    if a <= b:
        a, b = b, a
    return a if b == 0 else gcd(b, a - b)

print(gcd(42, 28)) # Output: 14
```

```
print(gcd(36, 14)) # Output: 2
print(gcd(13, 17)) # Output: 1
```

However, such an approach in the worst-case scenario still requires linear time in the size of the numbers: for example, when computing $\gcd(n, 1)$. The following lemma will be extensively employed in the so-called *Euclidean algorithm*, which will reduce the complexity down to logarithmic time.

Corollary 0.8. For any $a, b \in \mathbb{Z}$, we have $\gcd(a, b) = \gcd(b, a \bmod b)$.

Now, this is much better! Let us apply this idea again, but with the modulo operation instead of subtraction:

```
def gcd(a: int, b: int) -> int:
    return gcd(b, a % b) if b != 0 else a

print(gcd(42, 28)) # Output: 14
print(gcd(36, 14)) # Output: 2
print(gcd(13, 17)) # Output: 1
```

Notice that the algorithm can be easily implemented in a single line! Moreover, the complexity reduced down to logarithmic time: for computing $\gcd(a, b)$ we need $O(\log \max\{a, b\})$ time.

Example 0.4. Let us find the $\gcd(535, 230)$ manually. We will use the Euclidean algorithm for this purpose.

$$535 = 2 \cdot 230 + 75$$

$$230 = 3 \cdot 75 + 5$$

$$75 = 15 \cdot 5 + 0$$

Therefore, $\gcd(535, 230) = 5$.

While the greatest common divisor (GCD) focuses on finding the largest shared factor between two numbers, the least common multiple (LCM) deals with finding the smallest multiple that both numbers have in common. The LCM is particularly useful when we need to synchronize cycles or work with fractions.

Definition 0.9 (LCM). For any $a, b \in \mathbb{Z}$, the **least common multiple** $\text{lcm}(a, b)$ is defined as an integer $m \in \mathbb{N}$ such that:

1. $a \mid m$ and $b \mid m$
2. m is a minimal integer that satisfies the first condition

One might right the above definition more concisely:

$$\text{lcm}(a, b) = \min\{m \in \mathbb{N} : a \mid m \text{ and } b \mid m\}.$$

Example 0.5. Let us again find, but now lcm for few pairs of numbers.

- | | |
|-------------------------------|-------------------------------|
| 1. $\text{lcm}(12, 4) = 12$ | 3. $\text{lcm}(13, 18) = 234$ |
| 2. $\text{lcm}(13, 17) = 221$ | 4. $\text{lcm}(42, 28) = 84$ |

Lemma 0.10 (Least Common Multiple Properties).

1. We consider $\text{lcm}(a, 0)$ to be undefined.
2. $\text{lcm}(a, b) = a \Leftrightarrow b \mid a$.
3. If a and b are coprime, then $\text{lcm}(a, b) = a \cdot b$.
4. Any common divisor δ of a and b satisfies $\delta \mid \text{lcm}(a, b)$.
5. For any $c > 0$, we have $\text{lcm}(a \cdot c, c \cdot b) = c \cdot \text{lcm}(a, b)$.
6. Integers $\text{lcm}(a, b)/a$ and $\text{lcm}(a, b)/b$ are coprime.

Theorem 0.11. For any $a, b \in \mathbb{N}$, we have $\text{gcd}(a, b) \cdot \text{lcm}(a, b) = ab$.

One interpretation of the above theorem is that no additional algorithm is required for $\text{lcm}(a, b)$ if we already have an algorithm for $\text{gcd}(a, b)$. Indeed, we can simply use the formula $\text{lcm}(a, b) = ab/\text{gcd}(a, b)$ with the previously computed $\text{gcd}(a, b)$.

The reasonable question is how to generalize the gcd and lcm operations to more than two arguments. For that reason, we provide the following definition.

Definition 0.12 (GCD and LCM for multiple arguments). We define the **greatest common divisor** $\text{gcd}(a_1, a_2, \dots, a_n)$ and the **least common multiple** $\text{lcm}(a_1, a_2, \dots, a_n)$ for any set of integers $a_1, a_2, \dots, a_n \in \mathbb{Z}$ as follows:

$$\text{gcd}(a_1, a_2, \dots, a_n) = \max\{d \in \mathbb{N} : d \mid a_1, d \mid a_2, \dots, d \mid a_n\}.$$

$$\text{lcm}(a_1, a_2, \dots, a_n) = \min\{m \in \mathbb{N} : a_1 \mid m, a_2 \mid m, \dots, a_n \mid m\}.$$

Theorem 0.13 (Computational Properties Of GCD and LCM For Multiple Arguments.). The following two statements are true:

- $\forall a, b \in \mathbb{N} : \text{gcd}(a, b, c) = \text{gcd}(\text{gcd}(a, b), c) = \text{gcd}(a, \text{gcd}(a, b))$.
- $\forall a, b \in \mathbb{N} : \text{lcm}(a, b, c) = \text{lcm}(\text{lcm}(a, b), c) = \text{lcm}(a, \text{lcm}(a, b))$.

In conclusion, from these two theorems there is no conceptual necessity for specific algorithms for $\text{gcd}(\cdot)$ and $\text{lcm}(\cdot)$ when dealing with more than two arguments. It is worth noting that there are more effective algorithms for more than two arguments, but such a topic is well beyond the scope of this book.

0.2 Extended Euclidean algorithm

In this section, we will introduce the Extended Euclidean Algorithm and an important lemma related to the GCD. You might have reasonable question: why do we even need an extended version? One of the primary reasons is that this algorithm will help in finding inverse modular elements, introduced in the subsequent sections.

Lemma 0.14 (Bezout identity). For any two given integers $a, b \in \mathbb{N}$ with $d = \gcd(a, b)$ there exists such $u, v \in \mathbb{Z}$ that $d = au + bv$.

Example 0.6. It is evident that $\gcd(125, 93) = 1$, so we can find such $u, v \in \mathbb{Z}$ that $1 = 125 \cdot u + 93 \cdot v$. One such example is $u = 32$ and $v = -43$ since $1 = 125 \cdot 32 + 93 \cdot (-43)$. In this case, we call the tuple $(32, -43)$ as *Bezout coefficients*. We will explain and show how to find these coefficients concretely a bit later.

Corollary 0.15 (From Bezout identity).

1. Integers u and v are of different signs (excluding the case when either of Bezout coefficients is zero).
2. *Generalization for multiple integers:* Suppose $d = \gcd(a_1, a_2, \dots, a_n)$, then there exist such integers $u_1, u_2, \dots, u_n \in \mathbb{Z}$ that $d = \sum_{i=1}^n u_i a_i$.

The integers u and v are called **Bezout coefficients**. The first corollary can be understood intuitively: if all coefficients are non-negative, the result will be much larger than necessary. Similarly, if they are non-positive, the result must be negative, but the GCD was defined to be positive. The second consequence follows from the fact that we can decompose GCD, thus sequentially derive this sequence. Also note that we can find Bezout coefficients on each Euclidean algorithm step.

Now we introduce the **extended Euclidean algorithm**. The extended Euclidean algorithm, besides calculating the GCD itself, finds the Bezout coefficients efficiently (in the polylogarithmic time). For example, as we will see, for calculating the modular inverse element, one would rather use the extended Euclidean algorithm than the naive approach, which shows whether inverse exists at all. The time complexity of the Extended Euclidean algorithm is $O(\log \max\{a, b\})$ — the same as for the classic Euclidean algorithm.

Algorithm 1: Extended Euclidean algorithm

Input : $a, b \in \mathbb{N}$. Without loss of generality, $a \geq b$

Output: $(\gcd(a, b), u, v)$

```
1  $r_0 \leftarrow a; r_1 \leftarrow b$ 
2  $u_0 \leftarrow 1; u_1 \leftarrow 0$ 
3  $v_0 \leftarrow 0; v_1 \leftarrow 1$ 
4 while  $r_{i+1} \neq 0, i = 1, 2, \dots$  do
5    $q_i \leftarrow r_{i-1} \text{ div } r_i$ 
6    $u_{i+1} \leftarrow u_{i-1} - u_i q_i$ 
7    $v_{i+1} \leftarrow v_{i-1} - v_i q_i$ 
8    $r_{i+1} \leftarrow a u_{i+1} + b v_i$ 
9 end
10 return  $(r_i, u_i, v_i)$ 
```

Although we already know how the Extended Euclidean Algorithm works, this method is still not very intuitive. Let us consider an example to better understand how it works.

Example 0.7 (Extended Euclidean algorithm example).

First, we you need to find $d = \gcd(a, b)$. Then, knowing the sequence of expansions (quotient q_i at each step), we can find the Bezout coefficients.

1. $125 = 93 \cdot 1 + 32$
2. $93 = 32 \cdot 2 + 29$
3. $32 = 29 \cdot 1 + 3$
4. $29 = 3 \cdot 9 + 2$
5. $3 = 2 \cdot 1 + 1$
6. $2 = 1 \cdot 2 + 0$

i	0	1	2	3	4	5	6
q_i	$\times$	1	2	1	9	1	2
u_i	1	0	1	-2	3	-29	32
v_i	0	1	-1	3	-4	?	?

Here, each corresponding cell is calculated with the following formula:

$$u_{i-1} - q_i u_i = u_{i+1}$$

$$v_{i-1} - q_i v_i = v_{i+1}$$

Exercise. Finish this example by filling in the missed cells marked by ?. After finding v_6 , be sure to check yourself.

0.3 Prime numbers

Prime numbers are fundamental in mathematics due to their role of the building blocks of all natural numbers: in fact, every integer $n > 1$ can be uniquely factored into the product of primes, a concept known as the *Fundamental Theorem of Arithmetic*, which we introduce in the next section. This property makes primes central to number theory and cryptography in particular, where we will use them in finite fields extensively.

Definition 0.16. Number $p \in \mathbb{N}$ is called **prime** if and only if its only two positive divisors are 1 and n .

Definition 0.17. Number $n \in \mathbb{N}$ is **composite** iff there exists an integer $a \in \mathbb{N}$, which is not 1 or n , for which $a \mid n$. In other words, it is not prime.

Example 0.8. For example 3, 17, 19, 29, 71, 997, 7817, 104729 are all examples of prime numbers. In contrast, 8, 1001, 10000, 100000 are all examples of composite numbers.

One might ask: what to do with the number 1? We consider it to be neither prime nor composite. Another logical question is whether the set of prime numbers is finite or infinite. The next theorem settles this question.

Theorem 0.18 (Euclidean theorem). If $P \subset \mathbb{N}$ is a finite set, consisting of prime numbers, then there exists a prime number p' such that $p' \notin P$.

Proof Idea. We can set $p' := \prod_{p \in P} p + 1$. Although it is intuitively clear that p' is not divisible by any prime number in P , the formal proof requires a bit more effort.

Corollary 0.19. There exists an infinite number of primes.

Lemma 0.20. For all $n \in \mathbb{N}$, where n is composite, if there exists a minimal divisor $d > 1$ of n , then d is a prime number.

Consider a scenario where we have a large number and need to determine whether it is prime. How can this be achieved? In other words, what methods exist for primality testing? Several approaches are available. The most straightforward method is brute-force division, though it is impractical for large numbers. Probabilistic tests provide a more feasible alternative, though they introduce a small probability of error. In 2003, a significant breakthrough was made with the discovery of a deterministic primality test, which placed the problem within the P complexity class.

It is also important to note that various types of prime numbers have applications in different fields. Examples include Mersenne primes, factorial primes, Euclidean primes, and Fibonacci primes, among others. As an example, consider one of the most well-known categories (which you might encounter in the modern zero-knowledge protocols): Mersenne primes.

Definition 0.21 (Mersenne primes). The prime number of the form $2^p - 1$ is called **Mersenne prime**, where p is a prime number, usually notated as following $M_p = 2^p - 1$.

Example 0.9. For example, $2^7 - 1 = 127$ is a Mersenne prime, since 127 is a prime number. However, $2^{11} - 1 = 2047$ is not a Mersenne prime, since $2047 = 23 \cdot 89$. As of 2025, only 52 such numbers were found, the largest of which is $2^{136279841} - 1$.

Lemma 0.22. If the number M_p is prime, then so is p .

Mersenne primes, are important in both number theory and cryptography. They have unique mathematical properties that make them useful for testing primality and generating large prime numbers. Mersenne primes are also crucial in the construction of efficient algorithms for error correction in coding theory and for generating random numbers in cryptographic applications, etc.

Finally, we introduce the Dirichlet's theorem, which is sometimes used for prime generation in cryptographic systems or constructing special primes.

Theorem 0.23 (Dirichlet's theorem). For any $a, b \in \mathbb{N}$ if $\gcd(a, b) = 1$, then infinite primes number form of $am + b$ exists, where $m \in \mathbb{N}$.

In other words, every infinite arithmetic progression whose first term and difference are positive integers contains an infinite number of primes.

0.4 Fundamental Theorem of Arithmetic

The Fundamental Theorem of Arithmetic states that every integer greater than 1 can be uniquely factored into primes, which is crucial for understanding the structure of numbers. It plays a key role in areas like number theory, cryptography, and simplifying calculations involving divisibility.

Example 0.10. To see this fact in action, let us decompose an integer $n = 123456$ into a product of prime numbers.

$$123456 = 2^6 \cdot 3^1 \cdot 643^1$$

Before formulating the Fundamental Theorem of Arithmetic, let us consider the auxiliary Lemma 0.24.

Lemma 0.24 (Euclidean). If p is prime and $p \mid ab$, then $p \mid a$ or $p \mid b$.

As an exercise and to show where Bezout coefficients are used, let us prove

both this lemma and the subsequent theorem.

Proof. ► Let $p \mid ab$, but $p \nmid a$, then $\gcd(a, p) = 1$ because there are no common divisors. In which case, by Bezout identity [Definition 0.14](#), there exist such $u, v \in \mathbb{Z}$ that $au + pv = 1$. Let's multiply the left and right sides by b : $abu + pbv = b$, but $p \mid ab$ and $p \mid pb$, therefore their sum is also divisible by $p \mid abu + pbv$. ◀

Now, we are ready to introduce the central Number Theory theorem – Fundamental Theorem of Arithmetic.

Theorem 0.25 (Fundamental theorem of arithmetic). Any integer $n > 1$ can be decomposed in the unique way into a product of prime numbers:

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_t^{\alpha_t} = \prod_{j=1}^t p_j^{\alpha_j},$$

where $p_1, \dots, p_t$ are prime numbers and $\alpha_1, \dots, \alpha_t \in \mathbb{N}$.

Proof. ► We need to prove both existence and uniqueness. Since the existence is easy to prove, let us make the small exception and show it. For other proves (in particular, for the uniqueness case), see the literature provided.

Existence. Suppose the theorem statement is false and there exists n that does not have such a representation. Let n_0 be the smallest of them. If n_0 is prime, it can be represented as $p_1 = n_0$, $\alpha_1 = 1$, which satisfies the representation. Therefore, n_0 must be composite, which means $\exists a, b \in \mathbb{N}, 1 < a, b < n_0$ such that $n = ab$. Since n_0 is the smallest non-decomposable number and $a, b < n_0$, then a and b can be represented as

$$\begin{aligned} a &= p_1^{\alpha_1} \cdots p_t^{\alpha_t} \\ b &= q_1^{\beta_1} \cdots q_k^{\beta_k}. \end{aligned}$$

Therefore, $n_0 = ab = p_1^{\alpha_1} \cdots p_t^{\alpha_t} q_1^{\beta_1} \cdots q_k^{\beta_k}$. Thus, the contradiction, as n_0 is represented as the product of primes. ◀

Corollary 0.26. Suppose n is decomposed as in [Definition 0.25](#). Then,

1. If $d \mid n$, then $d = p_1^{\beta_1} p_2^{\beta_2} \cdots p_t^{\beta_t}$, where $0 \leq \beta_i \leq \alpha_i$.
2. Suppose $a = p_1^{\alpha_1} \cdots p_t^{\alpha_t}$ and $b = p_1^{\beta_1} \cdots p_t^{\beta_t}$. Then, the GCD can be evaluated as $\gcd(a, b) = \prod_{i=1}^t p_i^{\min\{\alpha_i, \beta_i\}}$.
3. Suppose $a = p_1^{\alpha_1} \cdots p_t^{\alpha_t}$ and $b = p_1^{\beta_1} \cdots p_t^{\beta_t}$. Then, the LCM can be evaluated as $\text{lcm}(a, b) = \prod_{i=1}^t p_i^{\max\{\alpha_i, \beta_i\}}$.
4. If $b \mid a$ and $c \mid a$ and $\gcd(b, c) = 1$, then $bc \mid a$.

The implications and applications of the Fundamental Theorem of Arithmetic are large and important, and the number of “obvious” ones are listed above.

Remark. Note that the problem of factorization, i.e., finding the decomposition of a number into prime factors, is in the NP class (of complexity) and is the basis of certain cryptosystems such as RSA. For stricter P/NP definitions, see ??.

0.5 Modular arithmetic

Now we are ready for practical applications of number theory, starting with modulo congruence. In this section, we will review the idea of congruence, learn how to use it, and explore its key properties. Understanding these properties will help simplify calculations and solve problems more efficiently in fields such as cryptography, algorithms, and number theory.

Definition 0.27. Two integers $a, b \in \mathbb{Z}$ are said to be **congruent** modulo $n \in \mathbb{N}$ iff $n \mid (a - b)$. We denote this by $a \equiv b \pmod{n}$.

Example 0.11. It easy to see that $26 \equiv 5 \pmod{7}$ since $7 \mid (26 - 5)$.

As always, we enumerate some key properties of congruence modulo n relation to better understand its nature. First, we consider the properties of reflexivity, symmetry, and transitivity to confirm that congruence modulo n is what is called an *equivalence relation*. For general equivalence relation definition, see ??.

Lemma 0.28 (Reflexivity, Symmetry, and Transitivity).

1. $a \equiv a \pmod{n}$
2. If $a \equiv b \pmod{n}$, then $b \equiv a \pmod{n}$
3. If $a \equiv b \pmod{n}$ and $b \equiv c \pmod{n}$, then $a \equiv c \pmod{n}$

In turn, the following lemma states that we can perform addition, subtraction, and multiplication on the congruent numbers modulo n similarly to the usual arithmetic.

Lemma 0.29. Suppose we have $a \equiv b \pmod{n}$ and $c \equiv d \pmod{n}$, then

1. $a \pm c \equiv b \pm d \pmod{n}$
2. $ac \equiv bd \pmod{n}$

Example 0.12. Let use see how the lemma works in practice.

1. $17 \times 26 \pmod{7} \equiv 3 \times 5 \equiv 15 \equiv 1 \pmod{7}$
2. $5 + 26 \pmod{7} \equiv 5 + 5 \equiv 10 \equiv 3 \pmod{7}$
3. $(-5) \times 13 \pmod{7} \equiv (7 - 5) \times 6 \equiv 2 \times 6 \equiv 12 \equiv 5 \pmod{7}$

It is clear that we can perform additions and multiplications modulo n as usual,

but what about division? Consider the following lemma.

Lemma 0.30. If $ca \equiv cb \pmod{n}$, $\gcd(c, n) = 1$, then $a \equiv b \pmod{n}$.

This lemma states that we can cancel out the same left and right parts respectively, but with a certain requirement: this part must be coprime to n . Consider the following example that shows why this requirement is mandatory.

Example 0.13. Let us try to simplify the following equation: $6 \equiv 2 \pmod{4}$. Suppose we divide both sides by 2, then we have the false statement $3 \equiv 1 \pmod{4}$. That being said, the requirement $\gcd(c, n) = 1$ is mandatory.

Remark. In particular, the example above shows why the division operation is fundamentally different from the regular real/rational numbers. We will see that, in fact, the arithmetic modulo n is not always what mathematicians call a **field**. You will see more details in ??.

Now, in no particular order, we enumerate the key properties of the congruence relation modulo n , which require special attention. Note that all these properties require the rigorous proof, but, as mentioned earlier, we will not delve into the details of the proofs in this book.

Lemma 0.31 (Key Congruence Relation Properties).

1. For any $k \neq 0$ and $a \equiv b \pmod{n}$, we have $ak \equiv bk \pmod{nk}$
2. If $a \equiv b \pmod{n}$, then we can divide both sides by $\gcd(a, b, n)$:

$$a/d \equiv b/d \pmod{n/d} \text{ with } d = \gcd(a, b, n)$$
3. If $d \mid n$ and $a \equiv b \pmod{n}$, then $a \equiv b \pmod{d}$
4. If $a \equiv b \pmod{n_i}$ for $i \in [k]$, then $a \equiv b \pmod{\text{lcm}(n_0, \dots, n_{k-1})}$
5. If $a \equiv b \pmod{n}$, then $\gcd(a, n) = \gcd(b, n)$

Finally, we introduce the concept of congruence classes and residue rings, which is the key concept in Cryptographic applications. In the subsequent discussion, we use notation $n\mathbb{Z}$ to denote the set of all multiples of n .

Definition 0.32. The congruence class or residues of k modulo n is defined as a set $k + n\mathbb{Z} = \{k + nt \mid t \in \mathbb{Z}\}$, sometimes denoted as $[k]$ or $[k]_n$.

Example 0.14. For example, the congruence classes $[0]_2, [1]_2$ modulo 2 are respectively even and odd numbers:

$$\begin{aligned} [0]_2 &= \{0, \pm 2, \pm 4, \pm 6, \pm 8, \dots\}, \\ [1]_2 &= \{1, \pm 3, \pm 5, \pm 7, \pm 9, \dots\}. \end{aligned}$$

In turn, the congruence class $[3]_5$ modulo 5 is the set of numbers that give the remainder 3 when divided by 5:

$$[3]_5 = \{3, 3 \pm 5, 3 \pm 10, 3 \pm 15, \dots\}.$$

Definition 0.33. The **complete residue system modulo n** (or residue ring) is a set of integers, where every integer is congruent to a unique member of the set modulo n , usually denoted as $\mathbb{Z}_n = \{0, 1, 2, \dots, n-1\}$. In more formal literature, the set is often denoted as $\mathbb{Z}/n\mathbb{Z} = \{[0], [1], \dots, [n-1]\}$.

Example 0.15. $\mathbb{Z}_5$ is a complete residue system modulo 5 and consists of the following elements: $\mathbb{Z}_5 = \{0, 1, 2, 3, 4\}$. Sometimes, the formal literature defines $\mathbb{Z}/5\mathbb{Z}$ to be the set of congruence classes modulo 5:

$$\mathbb{Z}/5\mathbb{Z} = \{\{5k : k \in \mathbb{Z}\}, \dots, \{5k + 4 : k \in \mathbb{Z}\}\}.$$

The aforementioned lemmas and definitions describe the properties of addition, subtraction, and multiplication. But what about division? To define the division (if such operation is valid at all), we need to consider the so-called *modular multiplicative inverse*, which is usually denoted as $a^{-1} \pmod{n}$, similarly to real/rational numbers.

Definition 0.34. A modular multiplicative inverse of an integer a modulo n is such an integer a^{-1} that satisfies $a \cdot a^{-1} \equiv a^{-1}a \equiv 1 \pmod{n}$.

Example 0.16. Let us find the modular multiplicative inverse of $a = 3$ modulo $n = 7$. We need to find such a^{-1} that satisfies $3 \cdot a^{-1} \equiv 1 \pmod{7}$. By simple brute force, we find that $3 \cdot 5 \equiv 1 \pmod{7}$, thus $3^{-1} = 5$.

Remark. The inverse (if exists) behaves similarly to the usual inverse over rationals/reals. For example, $(a^2)^{-1} = (a^{-1})^2$, which means, we can first find the inverse, then the squared value, and vice versa.

The question then is when does the number have the inverse? Consider the following theorem.

Theorem 0.35. The modular multiplicative inverse $a^{-1} \pmod{n}$ exists if and only if $\gcd(a, n) = 1$.

Based on Extended Euclidean [Algorithm 1](#) and [Definition 0.35](#), we build the algorithm that can verify an existence and find the inverse value.

Algorithm 2: Modular multiplicative inverse algorithm

Input : a, n **Output**: a^{-1}

```
1  $(d, u, v) \leftarrow \text{ExtendedEuclideanAlgorithm}(a, n)$  /* See Algorithm 1 */
2 if  $d \neq 1$  then
3   | return inverse does not exist.
4 end
5 return  $u$ 
```

Let us see the concrete implementation of the Extended Euclidean Algorithm and the Modular Multiplicative Inverse Algorithm in Python.

```
def extended_gcd(a: int, b: int) -> Tuple[int, int]:
    x0, x1, y0, y1 = 1, 0, 0, 1
    while b:
        q, a, b = a // b, b, a % b
        x0, x1 = x1, x0 - q * x1
        y0, y1 = y1, y0 - q * y1
    return a, x0, y0

def inverse(a: int, n: int) -> int:
    d, u, v = extended_gcd(a, n)
    if d != 1:
        raise ValueError('Inverse does not exist')
    return u

# Example usage
print(inverse(3, 7)) # Output: -2
print(inverse(12, 17)) # Output: -7
print(inverse(18, 24)) # Panics as gcd(18, 24) != 1
```

Let us verify manually that 12^{-1} modulo 17 was found correctly. Consider the following example.

Example 0.17. Let us find the modular multiplicative inverse of $a = 12$ modulo $n = 17$. First of all, we need to verify that the inverse exists, in other words whether $\text{gcd}(12, 17) = 1$ is true at all.

$$(1) \quad 17 = 1 \cdot 12 + 5$$

$$(2) \quad 12 = 2 \cdot 5 + 2$$

$$(3) \quad 5 = 2 \cdot 2 + 1$$

$$(4) \quad 2 = 1 \cdot 2 + 0$$

Thus, we have verified the inverse existence. In the previous example, we used the Extended Euclidean Algorithm 1 to find Bezout coefficients, but in this case, we will show another method. Let us iteratively go backwards from

equation (3) and substitute the result from the earlier step.

$$\begin{aligned} 1 &= 5 - 2 \cdot 2 = 5 - 2(12 - 2 \cdot 5) = 5 \cdot 5 - 2 \cdot 12 \\ &= 5(17 - 12 \cdot 1) - 2 \cdot 12 = 5 \cdot 17 - 7 \cdot 12. \end{aligned}$$

As you can see, we have found Bezout coefficients, $\gcd(12, 17) = 5 \cdot 17 - 7 \cdot 12$. Therefore, the inverse is -7 , which is the output of the algorithm.

Remark. Note that the complexity of this algorithm is the same as for **Algorithm 1**, which is $O(\log a \log n)$ bit operations. In real applications, typically $n > a$, so the complexity is simplified to $O(\log^2 n)$.

Definition 0.36. The **multiplicative group of integers** modulo n , denoted as $\mathbb{Z}_n^\times$, is a set of natural numbers that are coprime to n and less than n . In other words, $\mathbb{Z}_n^\times = \{a \in \mathbb{N} : \gcd(a, n) = 1\}$.

Example 0.18. For instance, $\mathbb{Z}_{11}^\times = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ since all naturals less than 11 are coprime to 11 (since 11 is prime). However, $\mathbb{Z}_{12}^\times = \{1, 5, 7, 11\}$, since integers 2, 3, 4, 6, 8, 9, 10 are not coprime to 12.

The $\mathbb{Z}_n^\times$ is a fundamental object in number theory and plays a crucial role in cryptography, forming the basis almost for every cryptographic primitive. The structure of the group $\mathbb{Z}_n^\times$ has a crucial meaning and has deep connections with algebraic structures: even the number of elements of $\mathbb{Z}_n^\times$ has the special name, which we define below.

Definition 0.37. Euler's Totient Function $\varphi(n)$ is the cardinality of the multiplication group of integers $\mathbb{Z}_n^\times$. In other words, $\varphi(n) = |\mathbb{Z}_n^\times|$.

Remark. The alternative Euler's totient function interpretation is following: $\varphi(n)$ counts all coprime integers with n in range $[1, n]$.

Remarkably, $\varphi(n)$ has a lot of curious properties, which we specify below.

Lemma 0.38 (Euler's totient function properties).

1. $\varphi(1) = 1$.
2. $\varphi(p) = p - 1$, where p is prime.
3. $\varphi(pq) = \varphi(p) \cdot \varphi(q)$, where p, q are primes.
4. $\varphi(p^\alpha) = p^\alpha - p^{\alpha-1}$, where p is prime.

With these properties, we can derive a general formula for Euler's totient function, knowing the prime factorization of n .

Corollary 0.39. For any number $n = p_1^{\alpha_1} p_2^{\alpha_2} \dots p_t^{\alpha_t}$ general formula for Euler's totient function is:

$$\varphi(n) = \prod_{i=1}^t (p_i^{\alpha_i} - p_i^{\alpha_i-1}) = n \prod_{i=1}^t \left(1 - \frac{1}{p_i}\right).$$

Example 0.19.

- $\varphi(107) = 106$, because 107 is prime.
- $\varphi(123) = \varphi(3 \cdot 41) = \varphi(3)\varphi(41) = 2 \cdot 40 = 80$
- $\varphi(729) = \varphi(3^6) = 3^6 - 3^5 = 486$

0.6 Chinese Remainder Theorem

The Chinese Remainder Theorem is a fundamental result in number theory that provides an efficient method for solving congruence systems. It allows one to decompose a complex problem with large integers into several simpler problems with smaller modules. This decomposition reduces the required computational complexity, especially when working with large numbers, and is widely used in areas such as modular arithmetic, cryptography, and algorithmic number theory.

Theorem 0.40 (Chinese Remainder Theorem). Suppose there is the following system

$$\begin{cases} x \equiv x_1 \pmod{n_1} \\ x \equiv x_2 \pmod{n_2} \\ \dots \\ x \equiv x_t \pmod{n_t} \end{cases}$$

where the $n_1, \dots, n_t$ are pairwise coprimes and $x_1, x_2, \dots, x_t$ are fixed numbers. Then, this system has a unique solution modulo $N = n_1 n_2 \dots n_t$.

In fact, we can easily find such a solution. Let $N_i = N/n_i$. Since $n_1, \dots, n_t$ are pairwise coprimes, we have $\gcd(N_i, n_i) = 1$ for each i . By the Extended Euclidean Algorithm, there exists an integer a_i such that $a_i N_i \equiv 1 \pmod{n_i}$. Thus, the solution is given by

$$x_0 = a_1 N_1 x_1 + a_2 N_2 x_2 + \dots + a_t N_t x_t \pmod{N}.$$

This expression provides the unique solution modulo N , where $N = n_1 n_2 \dots n_t$. Besides, using this idea, we can write an efficient algorithm to compute x_0 iteratively.

Example 0.20. Let us solve the following system of congruences.

$$\begin{cases} x \equiv 1 \pmod{5} \\ x \equiv 2 \pmod{7} \\ x \equiv 3 \pmod{11} \end{cases}$$

First of all, we find $N = 5 \times 7 \times 11 = 385$, then we compute N_i .

- $N_1 = 7 \times 11 = 77$
- $N_2 = 5 \times 11 = 55$
- $N_3 = 5 \times 7 = 35$

To simplify the calculations, we give inverses for granted: $a_1 = 3, a_2 = 6, a_3 = 6$. The solution is then given by $x_0 \equiv 3 \times 77 \times 1 + 6 \times 55 \times 2 + 6 \times 35 \times 3 \equiv 1521 \equiv 81 \pmod{385}$.

However, as you can notice, the solution is not unique in general. We can increase the value with $N = 385$. Thus, the final solutions are $x = 81 + 385k \in \mathbb{Z}$.

0.7 Euler's Theorem

Now we are ready to introduce one of the fundamental results in Number Theory. Euler's Theorem was discovered by the mathematician Leonhard Euler as a generalization of Fermat's Little Theorem.

Why is it so important? Euler's Theorem is a key result in number theory and cryptographic applications, such as RSA encryption, and so on. Essentially, it helps in understanding the structure of the multiplicative group of integers modulo n and gives a foundation for other important results in number theory. Besides it provides the simplification of large exponents in modular arithmetic.

Theorem 0.41 (Euler's). $a^{\varphi(n)} \equiv 1 \pmod{n}$ for any $a \in \mathbb{Z}_n^\times$.

► To prove this fact, we prove the following auxiliary lemma.

Lemma 0.42. Suppose $a \in \mathbb{Z}_n^\times$. Denote by $a\mathbb{Z}_n^\times = \{ax : x \in \mathbb{Z}_n^\times\}$. Then, $\mathbb{Z}_n^\times = a\mathbb{Z}_n^\times$. In other words, elementwise multiplication by a permutes the elements of $\mathbb{Z}_n^\times$.

Proof. ► Our statement is equivalent to claiming that the function $f : \mathbb{Z}_n^\times \rightarrow a\mathbb{Z}_n^\times$ defined as $f(x) = ax \pmod{n}$ is a **bijection**. To show this, we need to prove the following three statements:

1. **Correctness** is obvious since ax is in $a\mathbb{Z}_n^\times$ if x is in $\mathbb{Z}_n^\times$.
2. **Injectivity:** We need to prove that if $f(x_1) = f(x_2)$, then $x_1 = x_2 \in \mathbb{Z}_n^\times$. From the function definition, we have $ax_1 \equiv ax_2 \pmod{n}$. By the following **Definition 0.31**, we can cancel a modulo n as a is coprime to n . Thus, $x_1 \equiv x_2 \pmod{n}$.
3. **Surjectivity:** We need to prove that every element $y \in a\mathbb{Z}_n^\times$ has at least one

pre-image $f^{-1}(y)$. Indeed, in this case we have $ax \equiv y \pmod{n}$. Then, set $x := a^{-1}y \pmod{n}$. This expression is well-defined as a is coprime to n and has an inverse.

Therefore, f is bijective and thus $\mathbb{Z}_n^\times = a\mathbb{Z}_n^\times$. ◀

So we know, $a\mathbb{Z}_n^\times = \mathbb{Z}_n^\times$, how do we proceed? Notice, from the set equality follows the equality of the products of all elements. Let us write this down:

$$\prod_{i=1}^{\varphi(n)} x_i \equiv \prod_{i=1}^{\varphi(n)} ax_i \pmod{n} \implies \prod_{i=1}^{\varphi(n)} x_i \equiv a^{\varphi(n)} \prod_{i=1}^{\varphi(n)} x_i \pmod{n}$$

Since all x_i are coprime to n by definition, we can cancel out the product of all x_i from both sides, leading to the conclusion of Euler's theorem. ◀

Example 0.21. Let us consider how Euler's Theorem works by computing seemingly scary expression $29^{202} \pmod{13}$. By means of **Definition 0.41** we have $29^{\varphi(n)} \equiv 1 \pmod{13}$, where $\varphi(13) = 12$. Using this, let us try to simplify the expression,

$$\begin{aligned} 29^{202} \pmod{13} &\equiv \\ &\equiv 29^{12 \times 16 + 10} \pmod{13} \\ &\equiv \cancel{(29^{12})^{16}} 29^{10} \pmod{13} \\ &\equiv 29^{10} \pmod{13} \\ &\equiv 3^{10} \pmod{13} \\ &\equiv 59049 \pmod{13} \\ &\equiv 3 \pmod{13}. \end{aligned}$$

In other words, we significantly reduced the computational complexity by using the Euler's theorem. Now, let us consider one essential corollary of Euler's theorem, which is used in finite field arithmetic.

Corollary 0.43. If p is a prime number and a is not divisible by p , then $a^{p-1} \equiv 1 \pmod{p}$.

This result is commonly known as Fermat's Little Theorem. However, to maintain generality, we state the theorem in a more general form below.

Theorem 0.44 (Fermat's Little Theorem.). Let p be a prime number. For any integer a , the following holds: $a^p \equiv a \pmod{p}$.

The proof of Fermat's Little Theorem is a direct consequence of Euler's theorem. We can consider Euler's Theorem as a generalization of Fermat's Little Theorem. Moreover, this fact allows to find the modular multiplicative inverse of an integer a modulo prime number faster than using the Extended Euclidean Algorithm.

Corollary 0.45. For any integer a and prime number p , the modular multiplicative inverse $a^{-1} \pmod{p}$ is $a^{p-2} \pmod{p}$.

Proof. Notice that $a \cdot a^{p-2} = a^{p-1} \equiv 1 \pmod{p}$ from Definition 0.43. Note that computing $a^{p-2} \pmod{p}$ requires less than $2 \log_2 p$ modular operations using the fast exponentiation algorithm. In Python, implementation using in-built pow function looks extremely simple:

```
def inverse(a: int, p: int) -> int:
    return pow(a, p-2, p)

print(inverse(3, 5))    # Output: 2
print(inverse(7, 39))   # Output: 7
print(inverse(8, 17))   # Output: 15
```

0.8 Exercises

Exercise 1. Which of the following statements is **true**?

- a) Prime factorization of 30 is $30 = 5 \cdot 6$.
- b) Number $2^{10} + 1$ is prime.
- c) Suppose $n \mid m$. Then m^2/n might not be an integer for some n, m .
- d) Any natural number n equals to the product of its divisors.
- e) Any prime number p equals to the product of its divisors.

Exercise 2. Which of the following statements is **false**?

- a) $\gcd(p, q) = 1$ for any two prime numbers p and q .
- b) $\text{lcm}(p, q) = pq$ for any two primes p and q .
- c) $\gcd(n, m) = \gcd(m, n)$ for any integers n, m .
- d) $\text{lcm}(n+1, n-1) = n^2 - 1$ for any integer n .

Exercise 3. What the inverse element $5^{-1} \pmod{11}$?

- a) 5
- b) 7
- c) 9
- d) 10

Exercise 4. What element is not contained in $\mathbb{Z}_{21}^\times$?

- a) 2
- b) 5
- c) 15
- d) 16

Exercise 5. What sum of all integers n satisfying $(n+1) \mid (n^2+1)$?

- a) 1
- b) 2
- c) 3
- d) 4

Exercise 6. In the quadratic polynomial $f(x) = (x-p)^2 - p(x-q) + x^2 - qx$, p and q are *distinct* prime numbers both less than 10. This polynomial can be rewritten as $f(x) = 2(x-r_1)(x-r_2)$ where r_1 and r_2 are the roots of f . If both r_1 and r_2 are prime numbers, what is their sum $r_1 + r_2$?

a) 7

b) 8

c) 10

d) 11

Exercises 7-9. Tau Function

Define function $\tau(n) := \{\# \text{ of positive divisors of } n\}$. For example, the number 12 has 6 divisors: 1, 2, 3, 4, 6, 12, so $\tau(12) = 6$. In fact, following **Definition 0.25** notation, if $n = \prod_{j=1}^r p_j^{\alpha_j}$, then the following holds: $\tau(n) = \prod_{j=1}^r (\alpha_j + 1)$. For example, since $12 = 2^2 \cdot 3^1$ we have $\alpha_1 = 2$, $\alpha_2 = 1$ and therefore $\tau(12) = (2 + 1) \cdot (1 + 1) = 6$.

Exercise 7. Suppose that p is an arbitrary prime. What is $\tau(p^{100})$?

a) 100

b) 101

c) 102

d) 99

Exercise 8. Suppose for some integer n we have $\tau(n) = 7$. What is $\tau(n^2)$?

a) 2

b) 6

c) 12

d) 13

Exercise 9*. What is the smallest integer m , greater than n^2 for some $n \in \mathbb{N}$, for which $\tau(m)$ is odd?

a) $n^2 + n$ b) $(n + 1)^2$ c) $n^2 + 1$ d) n^3